LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

CONSTRUCTOR, DESTRUCTOR, DAN OVERLOADING PADA PHP



Disusun oleh :

NAMA	:	GNAYAS AL HARITS
NIM	:	2530521067
KELAS	:	3 TI C

UNIVERSITAS MUHAMMADIYAH SUKABUMI

FAKULTAS SAINS DAN TEKNOLOGI

PROGRAM STUDI TEKNIK INFORMATIKA

2026

1. Capaian dan Tujuan Praktikum

• Menjelaskan fungsi constructor, destructor, dan overloading dalam konteks PHP.

• Membuat dan mengakses objek dengan konstruktor berparameter dan nilai default.

• Menganalisis kapan destructor dipanggil dan membedakannya dari operasi Delete pada CRUD.

• Mengimplementasikan __get(), __set(), dan __call() untuk akses dinamis secara terkendali.

• Mengembangkan studi kasus mahasiswa dan menguji hasil melalui browser/localhost.

2. Prasyarat, Perangkat, dan Persiapan

Prasyarat: mahasiswa memahami class, object, property, method, access modifier, operator ->, serta studi kasus CRUD Mahasiswa dari Pertemuan 3.

Perangkat: PHP 8.x (XAMPP), Apache, VS Code (PHP Intelephense disarankan), dan browser.

Folder kerja: C:\xampp\htdocs\pbo_php\pertemuan_04.

Akses: http://localhost/pbo_php/pertemuan_04/.

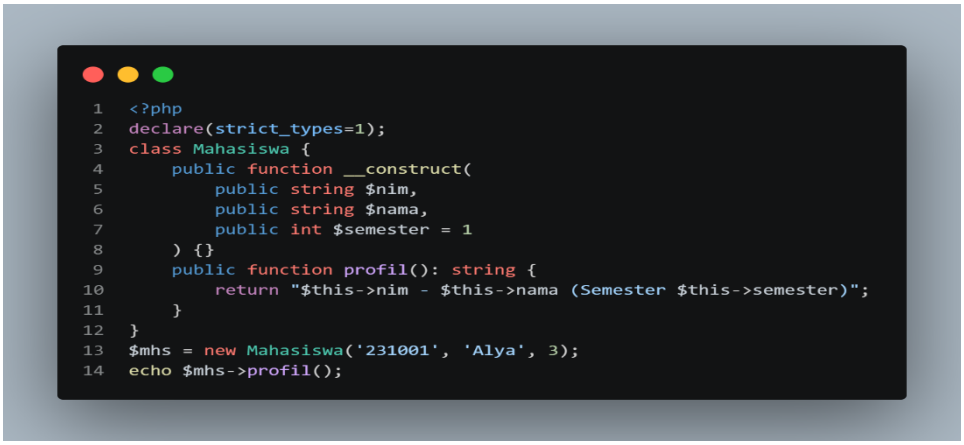
Sebelum praktikum: aktifkan Apache; buat folder kerja; pastikan file .php disimpan di dalam htdocs; gunakan file berbeda untuk tiap demonstrasi agar deklarasi class tidak bertabrakan.

3. Landasan Teori

3.1 Constructor: __construct()

Konstruktor adalah metode khusus yang dijalankan ketika objek dibuat dengan new. Konstruktor mengatur keadaan awal objek, misalnya NIM, nama, dan semester. Konstruktor tidak wajib ada; jika didefinisikan, argumen harus mengikuti parameter yang ditetapkan. Constructor property promotion (PHP 8+) dapat mempersingkat deklarasi dan pengisian properti.

Output: 231001 - Alya (Semester 3). Bahas: new membuat objek; __construct menerima nilai awal; $this menunjuk objek yang sedang digunakan; -> memanggil method.



```php
<?php
declare(strict_types=1);
class Mahasiswa {
    public function __construct(
        public string $nim,
        public string $nama,
        public int $semester = 1
    ) {}
    public function profil(): string {
        return "$this->nim - $this->nama (Semester $this->semester)";
    }
}
$mhs = new Mahasiswa('231001', 'Alya', 3);
echo $mhs->profil();
```

3.2 Destructor: __destruct()

Destructor adalah method khusus yang dipanggil saat objek dihancurkan, termasuk pada akhir eksekusi normal. unset($obj) menghapus variabel/referensi, tetapi tidak menjamin destructor langsung berjalan jika objek masih memiliki referensi lain. Urutan destructor pada akhir skrip tidak boleh dijadikan ketergantungan logika bisnis. Destructor bukan method untuk menghapus record mahasiswa dari Array/Session.



```php
<?php
class DemoSiklus {
    public function __construct() { echo "Object dibuat<br>"; }
    public function tampil(): void { echo "Method dipanggil<br>"; }
    public function __destruct() { echo "Destructor berjalan<br>"; }
}
$obj = new DemoSiklus();
$obj->tampil();
unset($obj);
echo "Program dilanjutkan";
```

Prediksi output sebelum menjalankan. Pada contoh tanpa referensi tambahan, destructor biasanya muncul setelah unset dan sebelum pesan terakhir. Jangan memakai destructor untuk menampilkan antarmuka atau menyimpan data penting secara andal.

3.3 Overloading dan Magic Methods

Konsep overloading pada PHP berbeda jauh dari bahasa Java atau C++. Di PHP, overloading bukan tentang membuat banyak fungsi bernama sama dengan parameter berbeda, melainkan mekanisme intersepsi dinamis ketika skrip mencoba membaca/menulis properti atau memanggil method yang tidak terdefinisi (atau bersifat *private*/*protected*). Fitur ini digerakkan oleh *magic methods* seperti __get(), __set(), __isset(), __unset(), dan __call(). Overloading disarankan hanya untuk atribut sekunder yang fleksibel, sedangkan atribut utama tetap wajib dideklarasikan secara eksplisit.

4. Source Code dan Hasil Percobaan

4.1 Percobaan A — Constructor dan Nilai Default (constructor.php)

```php
<?php
declare(strict_types=1);

class Mahasiswa {
    public function __construct(
        public string $nim,
        public string $nama,
        public int $semester = 1
    ) {}

    public function profil(): string {
        return "$this->nim - $this->nama (Semester $this->semester)";
    }
}

$mhs1 = new Mahasiswa('231001', 'Alya', 3);
echo $mhs1->profil() . "<br>";

$mhs2 = new Mahasiswa('231002', 'Rian'); // Menguji nilai default semester = 1
echo $mhs2->profil();
```

Output :



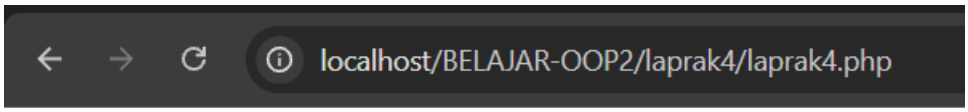
231001 - Alya (Semester 3)
231002 - Rian (Semester 1)

4.2 Percobaan B — Destructor dan Referensi Object (destructor.php)

```php
<?php
class DemoSiklus {
    public function __construct() { echo "Object dibuat<br>"; }
    public function tampil(): void { echo "Method dipanggil<br>"; }
    public function __destruct() { echo "Destructor berjalan<br>"; }
}

$obj = new DemoSiklus();
$obj->tampil();
unset($obj);
echo "Program dilanjutkan";
```

Output :



Object dibuat
Method dipanggil
Destructor berjalan
Program dilanjutkan

4.3 Percobaan C — Overloading Property (overloading_property.php)

```php
<?php
declare(strict_types=1);

class MahasiswaDinamis {
    private array $atributTambahan = [];

    public function __construct(public string $nim, public string $nama) {}

    public function __set(string $key, mixed $value): void {
        $diizinkan = ['hobi', 'asalSekolah', 'agama'];
        if (!in_array($key, $diizinkan, true)) {
            throw new InvalidArgumentException('Atribut tidak diizinkan');
        }
        $this->atributTambahan[$key] = $value;
    }

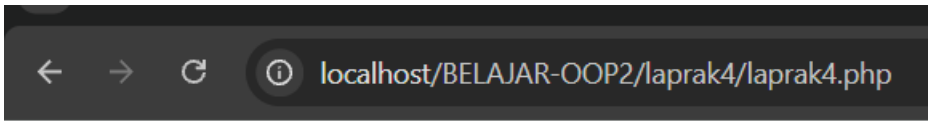
    public function __get(string $key): mixed {
        if (!array_key_exists($key, $this->atributTambahan)) {
            throw new OutOfBoundsException('Atribut tidak ditemukan');
        }
        return $this->atributTambahan[$key];
    }

    public function __isset(string $key): bool {
        return isset($this->atributTambahan[$key]);
    }

    public function __unset(string $key): void {
        unset($this->atributTambahan[$key]);
    }
}

$mhs = new MahasiswaDinamis('231001', 'Alya');
$mhs->hobi = 'Membaca';
echo htmlspecialchars($mhs->hobi, ENT_QUOTES, 'UTF-8');
```

Output :



Membaca

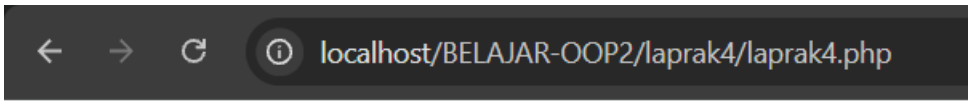
4.4 Percobaan D — Overloading Method dengan __call() (overloading_method.php)

```php
<?php
class LaporanMahasiswa {
    public function __construct(private string $nama) {}

    public function __call(string $method, array $args): mixed {
        if ($method === 'cetakRingkas' && count($args) === 0) {
            return 'Mahasiswa: ' . $this->nama;
        }
        throw new BadMethodCallException('Method tidak tersedia: ' . $method);
    }
}

$laporan = new LaporanMahasiswa('Alya');
echo $laporan->cetakRingkas();
```

Output :



Mahasiswa: Alya

5. Latihan Mandiri

Latihan 1: Constructor Mahasiswa dengan Parameter Fakultas Default

Soal: Modifikasi constructor Mahasiswa agar memiliki parameter fakultas dengan nilai default "Belum diisi"; buat dua objek dan bandingkan output.

Source Code :

```php
<?php
declare(strict_types=1);

class Mahasiswa {
    public function __construct(
        public string $nim,
        public string $nama,
        public int $semester = 1,
        public string $fakultas = "Belum diisi"
    ) {}

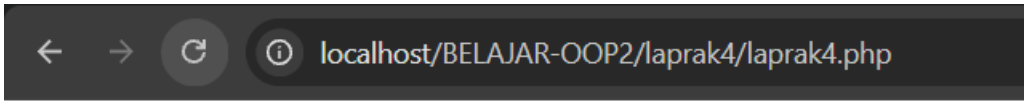
    public function getInfo(): string {
        return "NIM: $this->nim | Nama: $this->nama | Sem: $this->semester | Fakultas: $this->fakultas";
    }
}

// Objek 1: tanpa menyertakan argumen fakultas
$mhs1 = new Mahasiswa('2530101', 'Citra', 2);

// Objek 2: menyertakan argumen fakultas secara spesifik
$mhs2 = new Mahasiswa('2530102', 'Dharma', 4, 'Sains dan Teknologi');

echo "1. " . htmlspecialchars($mhs1->getInfo()) . "<br>";
echo "2. " . htmlspecialchars($mhs2->getInfo());
```

Output :



1. NIM: 2530101 | Nama: Citra | Sem: 2 | Fakultas: Belum diisi
2. NIM: 2530102 | Nama: Dharma | Sem: 4 | Fakultas: Sains dan Teknologi

Latihan 2: Class Buku dengan Constructor dan Method tampilInfo()

Soal: Buat class Buku dengan judul, penulis, dan tahun melalui constructor, serta method tampilInfo(); buat tiga objek.

Source Code:

```php
<?php
declare(strict_types=1);

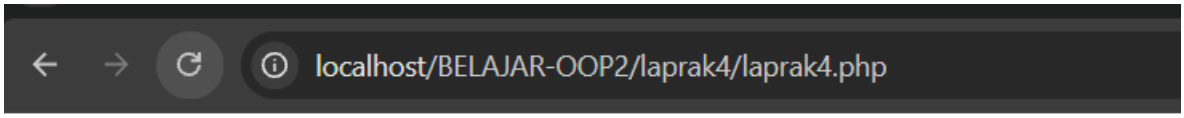
class Buku {
    public function __construct(
        public string $judul,
        public string $penulis,
        public int $tahun
    ) {}

    public function tampilInfo(): string {
        return "Judul: <i>{$this->judul}</i> | Penulis: {$this->penulis} | Tahun Terbit: {$this->tahun}";
    }
}

$bukuA = new Buku('Pemrograman Web Lanjut', 'Eko Prasetyo', 2022);
$bukuB = new Buku('Struktur Data & Algoritma', 'Fina Lestari', 2020);
$bukuC = new Buku('Jaringan Komputer Modern', 'Gita Permana', 2023);

echo "Buku 1: " . $bukuA->tampilInfo() . "<br>";
echo "Buku 2: " . $bukuB->tampilInfo() . "<br>";
echo "Buku 3: " . $bukuC->tampilInfo() . "<br>";
```

Output :



Buku 1: Judul: *Pemrograman Web Lanjut* | Penulis: Eko Prasetyo | Tahun Terbit: 2022
Buku 2: Judul: *Struktur Data & Algoritma* | Penulis: Fina Lestari | Tahun Terbit: 2020
Buku 3: Judul: *Jaringan Komputer Modern* | Penulis: Gita Permana | Tahun Terbit: 2023

Latihan 3: Destructor pada Class Buku dan Analisis Alias Object

Soal: Tambahkan destructor demonstrasi pada class Buku; bandingkan efek unset ketika masih ada alias object.

Source Code:

```php
<?php
declare(strict_types=1);

class Buku {
    public function __construct(
        public string $judul,
        public string $penulis,
        public int $tahun
    ) {
        echo "[INIT] Objek Buku '<b>{$this->judul}</b>' berhasil diinisialisasi.<br>";
    }

    public function detail(): string {
        return "'{$this->judul}' karya {$this->penulis} ({$this->tahun})";
    }

    public function __destruct() {
        echo "[DESTROY] Objek Buku '<b>{$this->judul}</b>' telah dihapus dari memori.<br>";
    }
}

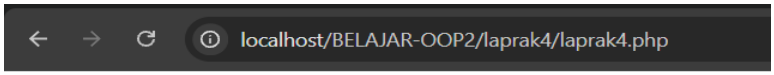
echo "=== SKENARIO 1: Unset Tanpa Alias ===<br>";
$buku1 = new Buku('PHP 8 Mastery', 'Heri', 2021);
unset($buku1); // Destructor dipanggil langsung
echo "Log: Setelah instruksi unset(\$buku1).<br><br>";

echo "=== SKENARIO 2: Unset Dengan Alias ===<br>";
$buku2 = new Buku('OOP Advanced', 'Irfan', 2024);
$refBuku2 = $buku2; // Menyimpan referensi kedua (alias)

unset($buku2); // Hanya menghapus referensi pertama
echo "Log: Perintah unset(\$buku2) selesai dijalankan.<br>";
echo "Akses via alias: " . htmlspecialchars($refBuku2->detail()) . "<br>";

unset($refBuku2); // Menghapus referensi terakhir, destructor baru dipanggil
echo "Log: Perintah unset(\$refBuku2) selesai dijalankan.<br>";
```

Output :



=== SKENARIO 1: Unset Tanpa Alias ===
[INIT] Objek Buku '**PHP 8 Mastery**' berhasil diinisialisasi.
[DESTROY] Objek Buku '**PHP 8 Mastery**' telah dihapus dari memori.
Log: Setelah instruksi unset($buku1).

=== SKENARIO 2: Unset Dengan Alias ===
[INIT] Objek Buku '**OOP Advanced**' berhasil diinisialisasi.
Log: Perintah unset($buku2) selesai dijalankan.
Akses via alias: 'OOP Advanced' karya Irfan (2024)
[DESTROY] Objek Buku '**OOP Advanced**' telah dihapus dari memori.
Log: Perintah unset($refBuku2) selesai dijalankan.

Latihan 4: Akses Atribut Dinamis dengan Magic Methods (__set, __get, __isset, __unset)

Soal: Tambahkan atribut dinamis “hobi” pada class latihan dan uji __set(), __get(), __isset(), dan __unset().

Source Code :

```php
<?php
declare(strict_types=1);

class MahasiswaDinamis {
    private array $dataAksesoris = [];

    public function __construct(public string $nim, public string $nama) {}

    public function __set(string $property, mixed $value): void {
        $allowed = ['hobi'];
        if (!in_array($property, $allowed, true)) {
            throw new InvalidArgumentException("Atribut '$property' dilarang untuk diisi.");
        }
        echo "[__set] Menyimpan nilai untuk '$property'<br>";
        $this->dataAksesoris[$property] = $value;
    }

    public function __get(string $property): mixed {
        echo "[__get] Mengakses nilai dari '$property'<br>";
        if (!array_key_exists($property, $this->dataAksesoris)) {
            throw new OutOfBoundsException("Properti '$property' tidak terdaftar.");
        }
        return $this->dataAksesoris[$property];
    }

    public function __isset(string $property): bool {
        echo "[__isset] Memeriksa keberadaan '$property'<br>";
        return isset($this->dataAksesoris[$property]);
    }

    public function __unset(string $property): void {
        echo "[__unset] Menghapus properti '$property'<br>";
        unset($this->dataAksesoris[$property]);
    }
}

$mhs = new MahasiswaDinamis('253099', 'Joni');

echo "<b>1. Pengujian __set():</b><br>";
$mhs->hobi = 'Catur';

echo "<br><b>2. Pengujian __get():</b><br>";
echo "Hobi Mahasiswa: " . htmlspecialchars($mhs->hobi) . "<br>";

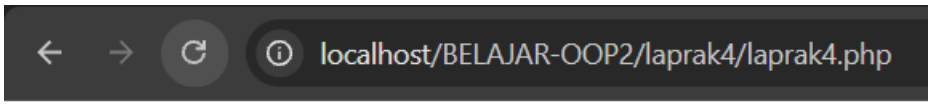
echo "<br><b>3. Pengujian __isset():</b><br>";
echo "Apakah 'hobi' ada? " . (isset($mhs->hobi) ? 'Ada' : 'Tidak Ada') . "<br>";

echo "<br><b>4. Pengujian __unset():</b><br>";
unset($mhs->hobi);
echo "Apakah 'hobi' ada setelah unset? " . (isset($mhs->hobi) ? 'Ada' : 'Tidak Ada') . "<br>";

echo "<br><b>5. Uji Exception __set() pada atribut dilarang:</b><br>";
try {
    $mhs->alamat = 'Sukabumi';
} catch (InvalidArgumentException $e) {
    echo "Tangkap Error: " . htmlspecialchars($e->getMessage()) . "<br>";
}

echo "<br><b>6. Uji Exception __get() setelah di-unset:</b><br>";
try {
    echo $mhs->hobi;
} catch (OutOfBoundsException $e) {
    echo "Tangkap Error: " . htmlspecialchars($e->getMessage()) . "<br>";
}
```

Output :



1. Pengujian __set():
[__set] Menyimpan nilai untuk 'hobi'

2. Pengujian __get():
[__get] Mengakses nilai dari 'hobi'
Hobi Mahasiswa: Catur

3. Pengujian __isset():
[__isset] Memeriksa keberadaan 'hobi'
Apakah 'hobi' ada? Ada

4. Pengujian __unset():
[__unset] Menghapus properti 'hobi'
[__isset] Memeriksa keberadaan 'hobi'
Apakah 'hobi' ada setelah unset? Tidak Ada

5. Uji Exception __set() pada atribut dilarang:
Tangkap Error: Atribut 'alamat' dilarang untuk diisi.

6. Uji Exception __get() setelah di-unset:
[__get] Mengakses nilai dari 'hobi'
Tangkap Error: Properti 'hobi' tidak terdaftar.

Latihan 5: Implementasi Penanganan Method Dinamis Menggunakan __call()

Soal: Buat __call() untuk menangani satu nama method yang diizinkan; uji pemanggilan method lain dan catat exception.

Source Code:

```php
<?php
declare(strict_types=1);

class LaporanMahasiswa {
    public function __construct(private string $nama) {}

    public function __call(string $method, array $arguments): mixed {
        // Hanya mengizinkan method 'cetakRingkas' tanpa argumen
        if ($method === 'cetakRingkas' && empty($arguments)) {
            return "Laporan Ringkas Mahasiswa: " . $this->nama;
        }
        throw new BadMethodCallException("Eksekusi gagal. Method '$method' tidak diizinkan atau argumen tidak sesuai.");
    }
}

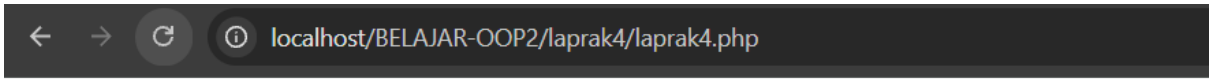
$lap = new LaporanMahasiswa('Kania');

echo "<b>1. Pemanggilan Method Valid:</b><br>";
echo htmlspecialchars($lap->cetakRingkas()) . "<br><br>";

echo "<b>2. Pemanggilan Method Tidak Terdaftar (cetakDetail):</b><br>";
try {
    echo $lap->cetakDetail();
} catch (BadMethodCallException $e) {
    echo "Exception Ditangkap: " . htmlspecialchars($e->getMessage()) . "<br><br>";
}

echo "<b>3. Pemanggilan Method Valid dengan Argumen Tambahan:</b><br>";
try {
    echo $lap->cetakRingkas('pdf');
} catch (BadMethodCallException $e) {
    echo "Exception Ditangkap: " . htmlspecialchars($e->getMessage()) . "<br>";
}
```

Output :



1. Pemanggilan Method Valid:
Laporan Ringkas Mahasiswa: Kania

2. Pemanggilan Method Tidak Terdaftar (cetakDetail):
Exception Ditangkap: Eksekusi gagal. Method 'cetakDetail' tidak diizinkan atau argumen tidak sesuai.

3. Pemanggilan Method Valid dengan Argumen Tambahan:
Exception Ditangkap: Eksekusi gagal. Method 'cetakRingkas' tidak diizinkan atau argumen tidak sesuai.

6. Tabel Hasil Pengujian (Uji Kesalahan dan Analisis)

Kasus uji	Gejala yang diharapkan	Analisis akar masalah	Hasil Pengujian Nyata
Jumlah parameter constructor salah	ArgumentCountError	Kontrak jumlah parameter pada __construct() tidak dipenuhi saat instansiasi objek.	Terjadi ArgumentCountError (*Too few arguments to function Mahasiswa::__construct()*) saat objek dibuat tanpa argumen wajib (misal: new Mahasiswa()).
Tipe parameter salah (strict_types=1)	TypeError	Tipe data argumen yang dioperkan tidak sesuai dengan deklarasi tipe pada parameter	Terjadi TypeError saat variabel $semester bertipe integer diisi dengan *string* non-numerik seperti "tiga".
Akses atribut dinamis dilarang	InvalidArgumentException	Atribut yang diakses tidak masuk dalam daftar array eksplisit ($diizinkan) pada method __set()	Terjadi InvalidArgumentException dengan pesan *"Atribut 'alamat' dilarang untuk diisi"* ketika mencoba menetapkan $mhs->alamat.
Method tidak dikenali pada __call()	BadMethodCallException	Pengguna memanggil method yang tidak terdefinisi di class dan tidak dilayani oleh kondisi __call()	Terjadi BadMethodCallException dengan pesan *"Method 'cetakDetail' tidak diizinkan..."*. Error ditangani secara terkendali tanpa menghentikan sistem
Unset satu referensi saat ada alias	Destructor tidak langsung berjalan	Objek masih tersimpan di memori karena nilai *reference counter* objek belum mencapai nol	Saat unset($buku2) dieksekusi, pesan destructor tidak muncul. Destructor baru berjalan setelah unset($refBuku2) dipanggil.

7. Analisis dan Pembahasan

Analisis Urutan Constructor – Method – Destructor

Berdasarkan eksperimen pada Percobaan B dan Latihan 3, urutan eksekusi siklus hidup (*lifecycle*) sebuah objek dapat dirinci sebagai berikut:

1. Constructor (__construct): Dieksekusi paling awal secara otomatis tepat pada saat kata kunci new dipanggil.
2. Method Biasa: Berjalan hanya ketika dipanggil secara eksplisit menggunakan operator -> di sepanjang alur program.
3. Destructor (__destruct): Berjalan paling akhir sewaktu *reference counter* dari objek bernilai nol (misal akibat unset() pada seluruh variabel perujuk) atau saat pemrosesan skrip PHP telah usai sepenuhnya.

Apabila sebuah objek memiliki alias (variabel referensi tambahan), pemanggilan unset() pada salah satu variabel hanya mengurangi *reference counter* dan tidak langsung menghancurkan objek. Hal ini mempertegas fakta bahwa destructor bertugas mengelola alokasi memori RAM internal, bukan untuk menghapus data record bisnis (seperti fungsi hapus() pada CRUD Mahasiswa/Session).

Analisis Pemicu Magic Methods

Magic methods beroperasi sebagai mekanisme intersepsi otomatis sewaktu skrip mengakses elemen *non-existent* atau *inaccessible*:

Magic Method	Pemicu Utama	Pengamatan Praktikum
__set($key, $val)	Penulisan properti dinamis/tidak terakses	Dipicu saat $mhs->hobi = 'Catur'. Mampu membatasi nama atribut lewat validasi
__get($key)	Pembacaan properti dinamis/tidak terakses	Dipicu saat echo $mhs->hobi. Melemparkan OutOfBoundsException jika nilai belum dieset
__isset($key)	Penggunaan isset() atau empty() pada properti dinamis	Mengecek ketersediaan kunci di dalam array penampung $dataAksesoris
__unset($key)	Eksekusi unset() pada properti dinamis	Menghapus elemen dari array internal penampung atribut dinamis.
_call($name, $args)	Pemanggilan method instance yang tidak terdefinisi.	Mengintersepsi pemanggilan method secara dinamis dan melempar BadMethodCallException jika method tidak terdaftar

Penggunaan magic methods sangat membantu untuk mendukung aspek fleksibilitas data (misal pencatatan hobi/asal sekolah). Namun, kelemahannya dapat mengaburkan struktur class utama serta menyembunyikan kesalahan ketik (*typo*) jika tidak diberi penanganan error/exception yang tegas.

8. Evaluasi dan Refleksi

1. **Apa perbedaan constructor dengan method biasa?**

Jawab: Constructor (__construct) berjalan secara otomatis saat objek baru diinstansiasi menggunakan perintah new, sedangkan method biasa harus dipanggil secara manual menggunakan operator -> setelah objek terbentuk.

2. **Mengapa nilai awal objek sebaiknya valid sejak dibuat?**

Jawab: Agar objek senantiasa berada dalam status (*state*) yang konsisten dan terhindar dari potensi bug/error sewaktu method-method di dalamnya diakses oleh bagian program lain.

3. **Apakah unset($obj) selalu langsung menjalankan destructor? Jelaskan.**

Jawab: Tidak selalu. unset($obj) hanya menghapus variabel penunjuk (referensi) tersebut. Jika objek masih dirujuk oleh variabel penunjuk lain (alias), destructor baru akan dieksekusi setelah seluruh variabel referensi tersebut dihapus atau eksekusi skrip selesai.

4. **Mengapa destructor tidak sama dengan method hapus() dalam CRUD?**

Jawab: Destructor berfungsi membersihkan keberadaan objek di memori RAM komputer saat objek dihancurkan. Sementara itu, method hapus() pada CRUD berfungsi menghapus record data permanen dari media penyimpanan bisnis (seperti Session, Array Data, atau Database).

5. **Apa perbedaan overloading PHP dengan method overloading di Java/C++?**

Jawab: Pada Java/C++, overloading berarti membuat beberapa method bernama sama di dalam satu class tetapi memiliki parameter/tipe data berbeda. Di PHP, overloading adalah mekanisme dinamis berbasis *magic methods* untuk mengintersepsi pembacaan, penulisan, atau pemanggilan properti/method yang tidak tersedia atau bersifat private/protected.

6. **Kapan penggunaan __get/__set bermanfaat, dan apa risikonya?**

Jawab:

- **Manfaat:** Berguna untuk menangani properti fleksibel/tambahan yang sifatnya opsional tanpa perlu mendeklarasikan banyak properti spesifik.
- **Risiko:** Struktur class menjadi kurang transparan, performa sedikit menurun, dan berisiko menyembunyikan kesalahan penulisan (*typo*) nama properti jika tidak dibatasi validasi.

9. Kesimpulan

1. Constructor memudahkan penetapan kondisi awal objek secara konsisten sejak diinstansiasi. Fitur *Constructor Property Promotion* pada PHP 8 membuat sintaksis kode jauh lebih ringkas.

2. Destructor mengatur penghancuran objek di level memori, bukan sarana penghapusan record bisnis pada CRUD. Penggunaan unset() hanya akan memicu destructor secara instan apabila objek tidak lagi memiliki referensi lain (*alias*).

3. Overloading pada PHP beroperasi melalui *magic methods* untuk mengintersepsi properti maupun method dinamis. Agar kualitas dan keamanan kode terjaga, penerapan magic methods harus selalu dilengkapi validasi dan penglemparan exception yang sesuai.

Link Github :

https://github.com/Gnayas/PBO-PERTEMUAN-4